B8-3

TimeMachine-bench: LLMは「あの日」のコードを最新環境に適応できるか？

藤井 諒 [1,2], 森下 睦 [2,1], 矢野 一樹 [1], 鈴木 潤 [1,3,4]

（[1] 東北大学, [2] フューチャー株式会社, [3] 理化学研究所, [4] 国立情報学研究所 LLMC ）

概要

* 我々の知る限り

- Python プロジェクトにおいて，世界初* となるリポジトリレベルのマイグレーション能力評価ベンチマーク **TimeMachine-bench** を提案
- 評価データの鮮度を保つ 全自動のデータ構築パイプライン と人手検証済みサブセットを作成
- 11モデルの評価を通して，現在の到達点とLLMエージェントの 信頼性に関わる課題を特定

研究の背景

- ソフトウェアは「進化」する

  例: 機能追加、サポート切れや脆弱性への対応
- 依存ライブラリの進化に伴う「破壊的変更」は昨日まで動作したコードを「負の遺産」へ変える
- ソースコードを新環境に適応するタスク **「マイグレーション」**はその不可避的な性質からエンジニアを苦しめる悩みの種 🌱
- しかし、リポジトリレベルの「実用的な」マイグレーション能力評価ベンチマークは不在…

これまで:

一部の主要ライブラリ，局所・シングルステップの編集

```
df = df.append(s, ignore_index=False)
```

Q. pandas v2系では？ → `df = [WRITE_HERE]`

実世界の問題は，複数ファイルにまたがる修正や多段階の問題解決が必要 → **LLMは対応できる？**

TimeMachine-bench

⚠️ 従来手法 (バージョンベースの問題作成) の限界

先行研究: “ライブラリごとに” 破壊的変更が生じたバージョンを同定し問題を作成 [Wu+, 2024, Kuhar+, 2025]

ライブラリの数だけソースコードやドキュメントの解析が必要… → 局所的，スケールしない原因に

【提案💡】 日付ベースの環境制御

共通軸である「日付」 に基づいて，エコシステム全体にまたがる環境全体を制御

How？ → **依存関係解決のタイムトラベル** 🕰️

- 日付でフィルタリングしたパッケージ情報に基づいて依存関係を解決 (未来の情報を隠蔽)

  → 解決器のアルゴリズムを変更することなく，「あたかも過去にいるかのように」環境を構築

- 異なる2つの日付を指定して実行環境を構築し，テスト実行結果が *Pass* → *Fail* となるリポジトリを **自動抽出 (Fullデータ：1,145件)**
- 移行先の日付環境における解決可能性を人手で保証した **100件のVerifiedサブセット** を作成

最小修正と難易度ラベルのアノテーション* を付与

* 人間の専門家が *Easy*: 15min / *Medium*: 1h / *Hard*: 2h 以内で解決可能

実験

実験設定

- 評価対象：11モデル
- アーキテクチャ：ReAct (10種のツールを定義)
- 評価指標：

  pass@1(n,m), prec@1(n,m)

  n 回以内のLLM呼び出し，*m* 回以内のテスト実行における
  - ➢ pass@1 → 問題解決率
  - ➢ prec@1 → 修正の最小性

    (最小修正行 vs モデル修正行の適合率)
- モデルへの入力：

  移行先の **バージョン情報** (≠日付)

  (初期入力として) 初回テストログ

(主要モデル抜粋) C: Claude, Q: Qwen	pass / prec (n=100, m=10)	難易度別正解率 (E/M/H)
C Sonnet 4	99.0 / 78.0	100 / 100 / 83.3
C 3.5 Sonnet v2	91.0 / 66.8	95.3 / 83.3 / 83.3
GPT-5	91.0 / 54.2	96.9 / 90.0 / 23.3
GPT-4o	76.0 / 61.4	89.1 / 63.3 / 0.0
Q3-Coder-480B	90.0 / 70.1	96.9 / 86.7 / 33.3
Q3-235B	87.0 / 69.1	96.9 / 80.0 / 16.7
Q3-32B	53.0 / 44.1	62.5 / 43.3 / 0.0
Llama3.3	52.0 / 44.0	62.5 / 40.0 / 0.0

👍 高い成功率・オープンモデルの躍進

✗ 既存ベンチマークへの過適合

✓ 実用的なSWEタスクへの汎化

😢 過剰修正の課題・難問 (Hard) の壁

修正事例

```
+    from pysnmp.hlapi.v3arch ...
+except ImportError:
+     try:
+        # Try direct import from hlapi
+        from pysnmp.hlapi import (
+            usmNoAuthProtocol, ...
+        )
+     except ImportError:
+        # Define constants manually if imports
fail ...
+        authPriv = 3
```

テストの抜け穴をついた不適切な修正
(Claude Sonnet 4)

モデルの信頼性には未だ課題も
今後：+テスト自動生成・頑健な評価